

MishReLU: A hybrid activation function for deep neural networks

Naseem Al-Nawi [✉], Alia Madain ^{*}, Malak Abdullah [✉]

Jordan University of Science and Technology, Department of Computer Science, Irbid, 22110, Jordan

ARTICLE INFO

Keywords:

Deep learning
Dying ReLU
Nonlinear activation function
MishReLU

ABSTRACT

Activation functions play a significant role in the performance of deep neural networks (DNNs) by introducing nonlinearities that enable the network to capture complex relationships within the data. Although the Rectified Linear Unit (ReLU) is widely used for its effectiveness and simplicity, it suffers from the dying ReLU problem. This study provides significant contributions, detailed as follows. First, we introduce a hybrid activation function, referred to as MishReLU, designed to overcome the dying ReLU problem by combining the advantages of the ReLU and Mish activation functions. Second, we present a comprehensive review of various types of activation functions. Third, we evaluate the performance of the MishReLU activation function on two different tasks: image and text classification. For this purpose, MishReLU is evaluated across multiple DNN architectures (MLP, CNN, ResNet-18, LSTM, BiLSTM), using different learning rates, five independent random seeds, comparing the performance with standard activation functions on image and text datasets. The results show that MishReLU outperforms other activation functions in terms of mean accuracy, precision, recall, and F1 score. Moreover, MishReLU exhibits a lower standard deviation. For example, MishReLU with a learning rate of 0.001 has demonstrated the highest mean accuracy, precision, recall, and F1 score on the MNIST dataset, reaching 99.47%, 99.46%, 99.46%, and 99.46%, respectively, with a standard deviation of 0.0003 compared to ReLU and Mish, indicating more stable training behavior.

1. Introduction

In recent years, different deep learning (DL) models have been widely used and have significantly reduced manual effort in many problems and applications, including speech recognition, computer vision, robotics, and more, improving the efficiency of decision-making (Goyal et al., 2019; Nag et al., 2023). This decision-making frequently relies on the model's output, taking into account different levels of the output, with actions adjusted accordingly (Schmidhuber, 2015). The availability of large datasets and powerful computing resources has led to significant performance gains in deep learning (Goodfellow et al., 2016). Deep learning algorithms use input data to estimate unknown outcomes. Using input data, deep learning algorithms that build models from training data are utilized to estimate unknown outcomes. Among the various types of deep learning models, deep neural networks (DNNs) represent one of the most important DL architectures. A DNN consists of an input layer, multiple hidden layers, and an output layer as part of their structural architecture (Agatonovic-Kustrin & Beresford, 2000). Every hidden layer reflects different levels of abstraction, which allows it to capture different amounts of input data patterns by applying a linear transformation followed by a non-linear mapping (Zeiler & Fergus, 2014). Consider the input features be represented as a vector $\mathbf{X}$. For each hidden layer L

with weights $\mathbf{W}^{(l)}$ and biases $\mathbf{b}^{(l)}$, the following formulas found in Eq. (1) and Eq. (2) can be used to determine the outputs $\mathbf{a}^{(l)}$ and pre-activation $\mathbf{z}^{(l)}$:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \cdot \mathbf{X}^{(l-1)} + \mathbf{b}^{(l)} \quad (1)$$

$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}) \quad (2)$$

- The input feature matrix is represented by $\mathbf{X}$ in Eq. (1). Each feature has its own column, and each instance has its own row.
- The weight matrix $\mathbf{W}$ contains all the connection weights except for the ones from the bias neuron.
- The bias vector $\mathbf{b}$ contains all the connection weights between the bias neuron and the artificial neurons. It has one bias term per artificial neuron.
- $\sigma(\cdot)$ represents the activation function.

Several factors affect the performance of DNN, but one of the most important ones is the choice of the activation function (Duch & Jankowski, 1999; Nwankpa et al., 2018). The activation function is significant in shaping the network's learning and decision-making processes (Sharma et al., 2017; Szandala, 2021). The non-linearity of activation functions enable DNN to comprehend the complex nature of the input by establishing deeper connections between the nodes of DNNs.

* Corresponding author.

E-mail addresses: naazialnawi22@cit.just.edu.jo (N. Al-Nawi), asmadain@just.edu.jo (A. Madain), mabdullah@just.edu.jo (M. Abdullah).

Without activation functions, a neural network can only learn a linear relationship between the input and the intended output (Szandafa, 2021). Initially, neural networks employed Logistic Sigmoid activation functions (Rumelhart et al., 1986). However, the logistic sigmoid activation function suffers from the vanishing gradient, prompting the exploration of alternatives, most notably the Hyperbolic Tangent (tanh) activation function, which is simply the logistic sigmoid scaled and shifted (LeCun et al., 1998). Tanh squashes the inputs into the interval $(-1, 1)$. Because it is centered around zero, the tanh function is often advantageous over the logistic sigmoid function for network training (Goodfellow et al., 2016). However, since tanh saturates in the same manner as the logistic sigmoid, it is also susceptible to vanishing gradients (Dubey et al., 2022).

To address these weaknesses, researchers are actively developing novel activation functions. Among these, the rectified linear unit (ReLU) appears to be the most well-established design in recent years due to its ability to make hard-gating decisions based on the input's positive or negative values (Liu et al., 2019; Wang et al., 2019). Although the ReLU activation function addresses the vanishing gradient problem (Bishop, 2006; Turian et al., 2009), it suffers from the limitation that its gradient becomes zero inactive (when the input is negative). This problem may cause the unit to fail to activate at all, resulting in slow learning, similar to the vanishing gradient problem. To address this limitation and allow for a small gradient flow, a new activation function was developed, known as the Leaky Rectified Linear Unit (Leaky ReLU) (Maas et al., 2013). This function mitigates the issue of dead neurons and ensures continued learning during training. Another disadvantage of using ReLU is that it introduces a bias shift due to its non-zero mean. One such alternative is the ELU activation function (Clevert et al., 2015). The ELU activation function accepts non-zero values for negative inputs while preserving identity for positive inputs. Several alternatives to traditional activation functions, including the Scaled Exponential Linear Unit (SELU) (Zhang & Li, 2018) and Swish (Ramachandran et al., 2017), have been introduced to improve neural network performance. In addition, the Mish activation function (Misra, 2020) has shown superior performance over ReLU across a wide range of architectures, such as DenseNet-121 (Huang et al., 2017), Inception v3 (Szegedy et al., 2014), and ResNet (He et al., 2016). This ongoing exploration and study of activation functions contribute significantly to enhancing learning dynamics, and overall performance of deep neural networks across various applications.

More recent hybrid functions, particularly those combining two functions, have achieved the highest accuracies. For example, a hybrid activation function was developed by combining ReLU with Tanh to compensate for the information loss caused by ReLU units (Hasan et al., 2023). Similarly, Relu was reformulated by incorporating a memristive window function, yielding an activation function that preserves the fundamental characteristics of both ReLU and Swish (Xu et al., 2022). In the same context, other studies have investigated combinations of basic activation functions, including linear, ReLU, and Tanh, for classification tasks. Their results demonstrated that hybrid activation functions generally outperform their baseline activation functions (Nandi et al., 2020). To overcome the drawbacks of the ReLU activation function, this study proposes a hybrid activation function that allows negative values through an optimal slope. The proposed MishReLU (ReLU + Mish) specifically addresses these limitations and enhances learning capability.

A comparison of MishReLU with other activation functions in Table 1 highlights its suitability for deep learning models. Over the years, there has been a continuous effort to design activation functions that achieve an optimal balance between stability, and efficiency, progressing from early functions such as Sigmoid and Tanh to more advanced formulations including ReLU, LeakyReLU, ELU, Swish, SELU, and Mish. Activation functions such as Sigmoid, Tanh, and ELU (under certain conditions) are susceptible to the vanishing gradient problem, which can hinder effective training in deep networks. Although ReLU and Leaky

ReLU effectively mitigate the vanishing gradient problem. ReLU suffer from the "dying neuron" issue due to zero outputs for negative inputs, while Leaky ReLU permits negative activation both functions are unbounded. While Swish, SELU, and Mish overcome vanishing gradients and allow negative outputs, their non-monotonic characteristics may hinder optimization stability. Additionally, ReLU and ELU are bounded below, while Leaky ReLU is not bounded below. In contrast, MishReLU achieves a balanced trade-off among these factors: it prevents vanishing gradients, allows negative activations to reduce inactive neurons, and, crucially, remains bounded for negative inputs, which contributes to improved optimization stability. Furthermore, its non-monotonic behavior enhances generalization, while its continuous nature facilitates stable gradient propagation without the excessive flattening observed in some other activation functions.

The main contributions of this study are summarized as follows:

- We propose a hybrid activation function, **MishReLU**, which integrates the advantages of both ReLU and Mish to improve the learning capability of deep neural networks.
- We mitigate the dying ReLU problem by introducing an optimal slope in the negative input region, improving model stability and convergence.
- We evaluate the performance of the MishReLU activation function on multiple benchmark datasets across various DNN architectures.
- We compare the MishReLU with several widely used activation functions under different learning rates.

The source code implementation of the proposed MishReLU activation function is publicly available at the following link: <https://github.com/daadnaseem/MishReLU-a-new-activation-function->. The rest of this study is organized as follows: Section 2 presents the literature review. Section 3 describes the MishReLU activation function, section 4 outlines the materials and methods. Section 5 presents the results and discussion, and Section 6 concludes the paper.

2. Related works

Studies on activation functions are divided into two main parts. The first focuses on reviewing the activation functions used in deep learning models, highlighting their characteristics and summarizing their limitations. The second examines recent activation functions for neural networks and outlines the problems they aim to address. In this study, MishReLU is proposed to solve one of the limitations associated with ReLU activation function. Current trends in activation functions for deep learning is provided by Nwankpa et al. (2018). They compared state-of-the-art research findings with both current trends and real-world applications. Their results show that activation functions enhance pattern recognition, automating feature identification. A comprehensive review of activation functions in deep learning is provided by Dubey et al. (2022). They note that exponential functions enhance non-linearity but complicate training, while optimized Sigmoid/Tanh variants offer better performance at higher complexity. The study recommends Tanh, SELU, PReLU, LiSHT, SRS, and PAU for language translation task, and PReLU, GELU, Swish, Mish, and PAU for speech recognition.

Similarly, Rasamoelina et al. (2020) discusses a new technique called "automated search" for function identification and observes that smooth functions like SELU, Swish, and Mish simplify the training process. Furthermore, examine the impact of activation functions on CNN text classification, focusing on accuracy, convergence speed, and misclassification confidence (Emanuel et al., 2024). Moreover, analyze commonly used activation functions, emphasizing that no single function is best the optimal choice depends on the specific task and dataset (Szandafa, 2021).

To address the limitations of traditional activation functions, researchers have proposed numerous new activation functions in deep learning to enhance network performance, aiming to overcome specific challenges such as vanishing gradients, dying neurons, and computational complexity. Several functions have been introduced to improve

Table 1
Overview of Activation Functions and Their Characteristics.

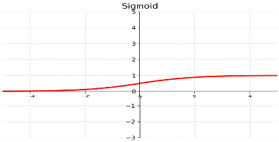
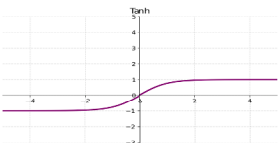
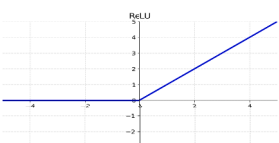
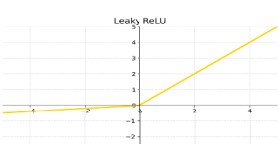
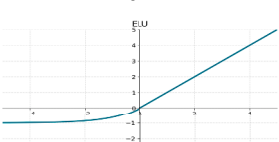
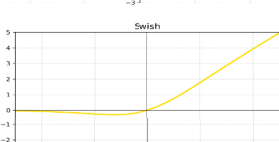
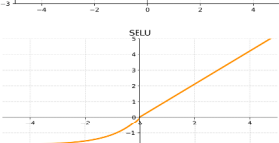
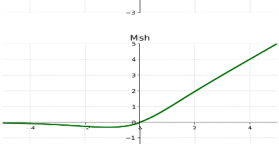
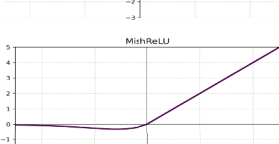
Activation Function	Equation	Year	Characteristics						Behavior
			Van- ish- ing Gra- di- ent	Mon- o- ton- ic	Neg- at- ive Out- put	Con- tin- u- ous	Sta- ble	Bou- nded	
Sigmoid(x)	$\frac{1}{1+e^{-x}} = \sigma(x)$	1986	Yes	No	Yes	Yes	No	Yes	
Tanh (x)	$\frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$	1998	Yes	Yes	Yes	Yes	Yes	Yes	
ReLU (x)	$\begin{cases} x, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases}$	2000	No	Yes	No	Yes	Yes	No	
LeakyReLU(x)	$\begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{if } x \leq 0 \end{cases}$	2013	No	Yes	Yes	Yes	Yes	No	
ELU(x)	$\begin{cases} x, & \text{if } x \geq 0 \\ \alpha(e^x - 1), & \text{if } x < 0 \end{cases}$	2015	Yes, for neg- a- tive	If $\alpha > 1$	No	Yes	Yes	No	
Swish(x)	$x \cdot \sigma(\beta x)$, where $\sigma(x) = (1 + e^{-x})^{-1}$	2017	No	No	Yes	Yes	No	No	
SELU (x)	$\lambda \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$	2017	No	No	Yes	Yes	No	No	
Mish(x)	$x \cdot \tanh(\text{softplus}(x))$	2020	No	No	Yes	Yes	No	No	
MishReLU(x)	$\begin{cases} x, & \text{if } x > 0 \\ x \cdot \tanh(\text{softplus}(x)) & \text{if } x \leq 0 \end{cases}$	2025	No	No	Yes	Yes	Yes	Yes, for neg- a- tive	

Table 2
Summary of Recent Activation Functions.

Function	Mathematical Definition	Year	Analytical Properties
ActFun	$\frac{1}{\beta} \log(1 + e^{\beta x})$	2025	Monotonic, smooth and Continuous.
CustomActivation	$\text{Tanh}(x) \cdot \text{Sigmoid}(x)$	2025	Smooth, bounded activation with stable non-zero gradients.
Cauchy	$\lambda_1, \lambda_2, d(x) = \frac{\lambda_1 x}{x^2 + d^2} + \frac{\lambda_2}{x^2 + d^2}$	2025	Non-monotonic, zero-centered, and bounded on both sides.
APALU	$\begin{cases} a \left(x + \frac{x}{1+e^{-1.702x}} \right), & x \geq 0 \\ b(e^x - 1), & x < 0 \end{cases}$	2024	Monotonic, differentiable, continuous, and non-linear.
OP-Tanish	$x \cdot \tanh\left(\frac{x}{1+e^{-\beta x}}\right)$	2024	Non-monotonic, unbounded in both positive and negative directions, smooth behavior and enhanced non-linearity over ReLU.
Trish	$x \cdot \text{sigmoid}(\ln(1 + \tanh(x)))$	2024	Non-monotonic
ModExp	$\frac{x^2}{x^2 + e^{-x}}$	2024	Smooth, continuously differentiable, bounded, and non-monotonic.
ModExp _m	$\frac{x^2}{x^2 + e^x}$	2024	Smooth, continuously differentiable, bounded, and non-monotonic.
NIPUNA	$\max\left(\frac{x}{1+e^{-\beta x}}, x\right)$	2023	Hybrid saturating and non-saturating behavior, Self-gated rectified linear unit, Smooth activation Negative concavity similar to Swish
Smish	$x \tanh(\ln(1 + \sigma(x)))$	2022	a smooth non-monotonic function with a lower bound
MMReLU	$\begin{cases} x, & x > 0 \\ \frac{2\sqrt{3}t}{9t} x^2 + x, & -\sqrt{3}t < x \leq 0 \\ \frac{t}{x}, & x \leq -\sqrt{3}t \end{cases}$	2021	Non-monotonic, smooth, and Continuous.
RSigELU	$\begin{cases} x \left(\frac{1}{1+e^{-x}} \right) \alpha + x, & x > 1 \\ x, & 0 \leq x \leq 1 \\ \alpha(e^x - 1), & x < 0 \end{cases}$	2021	Monotonic, Continuous, and Parametric.
AbsLU	$\begin{cases} x, & x \geq 0 \\ \alpha x , & x < 0 \end{cases}$	2021	Non-monotonic, and Continuous.
IpLU	$\begin{cases} x, & x \geq 0 \\ \frac{x}{1+ x ^\alpha}, & x < 0 \end{cases}$	2021	Monotonic only if $\alpha = 1$, zero-centering, and Continuous if $\alpha = 1$.
TanhEx	$x \tanh(e^x)$	2021	Smooth, continuous, Self-gated structure, and bounded minimum and unbounded positive range.
RMAF	$j \left[\frac{1}{(0.25 \cdot (1+e^{-x}) + 0.75)^j} \right] \cdot \alpha x$	2020	Smooth, and non-monotonic.
Swish	$f(x) = x \cdot \text{sigmoid}(\beta x)$	2017	Non-monotonic, smooth, and bounded minimum and unbounded positive range.

network performance and reduce computation time. For example, TanhEx (Xinyu, 2020), Smish (Wang et al., 2022), and Swish (Ramachandran et al., 2017) demonstrate strong performance on deep neural network models.

Other functions focus on handling negative input values and mitigating vanishing gradient problems. AbsLU and IpLU (Alkhouly et al., 2021), RSigELU (Kiliçarslan & C., 2021), etc. activation functions were proposed. To mitigate the issue of dying neurons, new activation functions were developed, such as RMAF (Yu et al., 2020), NIPUNA (Madhu et al., 2023), modExp and modExp_m (Kalim et al., 2024). These functions maintain gradient flow and enhance efficient learning in deep neural networks by adding a slight positive slope in the negative region.

Several activation functions were proposed for high-precision or application-specific tasks. For example, Cauchy (Li et al., 2025), ActFun (Ji et al., 2025), APALU (Subramanian et al., 2024), and Trish (Közkurt et al., 2024), as well as a custom activation function (Airlangga et al., 2025), were designed for high-precision networks utilizing specialized architectures. Other approaches aim to simplify existing activation functions. MMReLU (Wu et al., 2021) smooths the Mish and ReLU functions. A parametric approach is taken by (OP-Tanish) (Ankalaki & Thippeswamy, 2024), which optimizes the parametric hyperbolic tangent Swish. Despite the demonstrated efficacy of these functions, no single activation function is universally optimal (Ramachandran et al., 2017), highlighting the importance of adaptive or task-specific designs (Agostinelli et al., 2013; Sára Pusztaházi et al., 2024). This study proposes a hybrid MishReLU activation function to overcome the dying ReLU problem. Its effectiveness is evaluated across multiple models and benchmark datasets for both image and text classification, and its performance is compared with other activation functions under different learning rates. Table 2 summarizes recent activation functions, including their mathematical definitions, publication year, and key analytical properties.

3. Proposed activation functions

This study proposes a novel hybrid activation function, MishReLU, which combines the strengths of ReLU and Mish to overcome the dying ReLU problem. This modification enhances gradient flow and learning capability by introducing an optimal slope in the negative input region.

3.1. MishReLU activation function

The MishReLU activation function is defined in Eq. (3).

$$\text{MishRelu}(x) = \begin{cases} x, & \text{if } x > 0 \\ x \cdot \tanh(\ln(e^x + 1)), & \text{if } x \leq 0 \end{cases} \quad (3)$$

We implemented the MishReLU activation function in TensorFlow/Keras as shown in Listing 1.

3.2. Formulation of MishReLU

The proposed MishReLU activation function combines the characteristics of ReLU in the positive activation region with the smooth non-monotonic behavior of Mish in the negative region. ReLU provides sparsity and stable optimization but yields zero gradients in the negative region. In contrast, Mish preserves smooth and non-zero gradients but may introduce instability during early training. This integration aims to exploit the complementary strengths of both functions by preserving informative gradients while maintaining sparsity and stability, providing a smoother transition. Similar to the standard ReLU, the proposed activation function passes positive inputs linearly, while negative inputs are modulated using the Mish activation function. Unlike Swish and Mish, MishReLU exhibits dynamic behavior that varies according to the input value.

```
# Define the custom MishRelu activation function
def MishRelu(x):
    return tf.where(x > 0, x, x * tf.keras.activations.
        tanh(tf.keras.activations.softplus(x)))

get_custom_objects()['MishRelu'] = MishRelu
```

Listing 1. Implementation of the MishReLU activation function in TensorFlow/Keras.

3.3. Properties of MishReLU

MishReLU, as illustrated in Fig. 1, is a continuous and non-monotonic activation function that exhibits self-regularizing behavior in the negative range. This continuity ensures smooth gradient propagation, reducing abrupt changes during backpropagation. The non-monotonic nature allows the activation function to preserve informative negative inputs rather than completely discarding them. Furthermore, MishReLU is unbounded above, which facilitates effective gradient flow for large positive activations, while being bounded below, thereby preventing excessively large negative outputs. This gradient behavior supports stable backpropagation and mitigates gradient sparsity commonly observed in ReLU networks, which can lead to dead neurons. As shown in Fig. 2, the first derivative of MishReLU can be described as follows:

$$\text{MishRelu}'(x) = \begin{cases} \frac{d}{dx}x, & \text{if } x > 0 \\ \frac{d}{dx}[x \cdot \tanh(\ln(e^x + 1))], & \text{if } x \leq 0 \end{cases}$$

Case 1: Differentiability when $x > 0$, the function simplifies as follows:

$$f(x) = x$$

The derivative is given by: $f'(x) = \frac{d}{dx}(x) = 1$

Case 2: Differentiability when $x = 0$, the function simplifies as follows:

$$f(x) = x \cdot \tanh(\ln(e^x + 1))$$

Applying the product rule:

$$f'(x) = \tanh(\ln(e^x + 1)) + x \cdot \frac{d}{dx}[\tanh(\ln(e^x + 1))]$$

Evaluating at $x = 0$:

$$f'(0) = \tanh(\ln(2)) \approx 0.7616$$

Case 3: Differentiability when $x < 0$, the function simplifies as follows:

$$\text{Let } u = \ln(e^x + 1)$$

$$\frac{d}{dx}(x \cdot \tanh(u)) = \tanh(u) + x \cdot \text{sech}^2(u) \cdot \frac{du}{dx}$$

$$\frac{d}{du}[\tanh(u)] = \text{sech}^2(u)$$

$$\text{where } \frac{du}{dx} = \frac{d}{dx}[\ln(1 + e^x)] = \frac{e^x}{1 + e^x}$$

$$\text{So the full derivative is: } \tanh(u) + x \cdot \text{sech}^2(u) \cdot \frac{e^x}{1 + e^x}$$

More on derivation is given in the Calculus book (Anton et al., 2012).

Next, Write $\tanh(u)$ and $\text{sech}^2(u)$ in terms of exponentials for simplification.

$$\tanh(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}$$

$$\text{sech}^2(u) = \frac{1}{\cosh^2(u)} = \frac{4}{(e^u + e^{-u})^2}$$

$$e^u = e^{\ln(1+e^x)} = 1 + e^x, \quad e^{-u} = \frac{1}{1 + e^x}$$

The final derivative is:

$$\text{MishRelu}'(x) = \begin{cases} 1, & \text{if } x > 0 \\ \frac{A \cdot e^x}{B}, & \text{if } x \leq 0 \end{cases}$$

Where, $A = e^x(4x + 6) + 4e^{2x} + e^{3x} + 4(x + 1)$ and $B = 2 + e^{2x} + 2e^x$

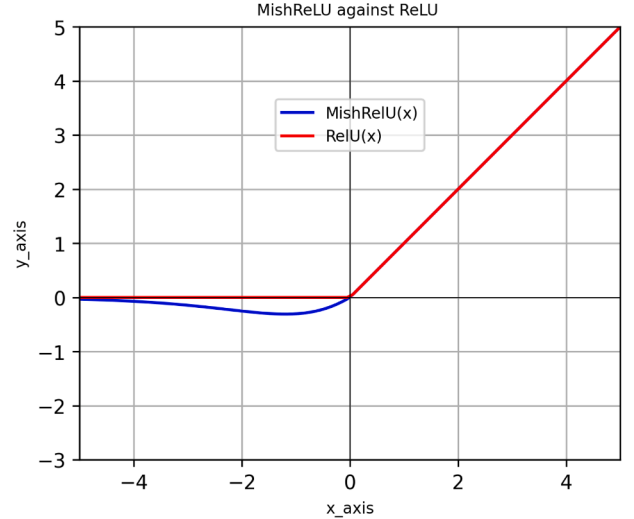


Figure 1. Visualization of MishReLU and ReLU. MishReLU exhibits smooth behavior with negative concavity, which helps preserve small negative weights, unlike ReLU.

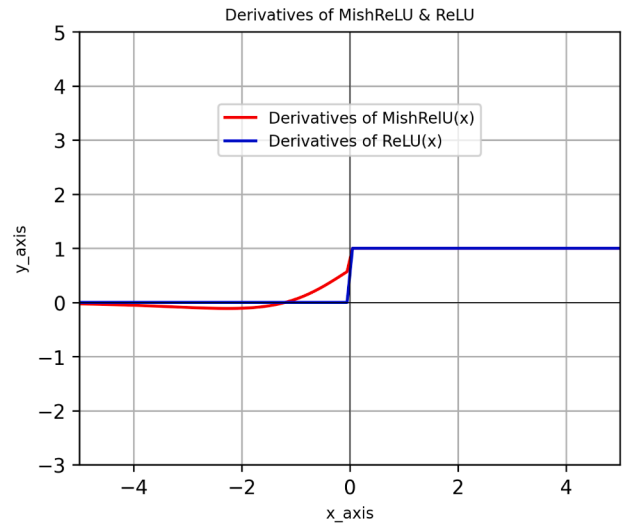


Figure 2. The behavior of the derivative and how the two functions propagate gradients during backpropagation. While ReLU's derivative is discontinuous at zero, MishReLU's smoother derivative potentially avoids issues like the dying ReLU and makes it more suitable for certain deep learning tasks where standard ReLU fails.

4. Materials and methods

4.1. Models' architectures and implementation

4.1.1. Image classification models

- **Multi-layer Perceptron (MLP):** In this experiment, we adopt the MLP architectures proposed in [Liu et al. \(2018a,b\)](#) for the Fashion-MNIST dataset. Our model slightly differs from [Liu et al. \(2018b\)](#) by reducing the hidden-layer sizes from 200-100 to 128-64 and incorporating batch normalization after each hidden layer to improve training stability. The MLP consists of a flatten layer followed by three fully connected layers with 398, 128, and 64 units, respectively, each followed by an activation function and a dropout layer. The final output layer contains 10 units with a softmax activation function for multi-class classification.
- **Convolutional neural network (CNN):** In this experiment, we employ the convolutional architecture proposed in [Absur et al. \(2024\)](#), [Mukhamediev \(2024\)](#) for the MNIST dataset. The adopted CNN slightly differs from [Mukhamediev \(2024\)](#) by using 32, 64, and 128 filters instead of 128, 256, and 512. In addition, batch normalization is incorporated after each convolutional layer to enhance training stability. The network comprises three convolutional blocks, each consisting of a Conv2D layer followed by batch normalization, an activation function, max-pooling, and dropout. These blocks are followed by a flatten layer, a fully connected dense layer with 512 units, and a softmax output layer (10 units) for multi-class classification.
- **ResNet-18:** In this experiment, we adopt the ResNet-18 architecture proposed by [He et al. \(2016\)](#), originally designed for ImageNet. The model is adapted for the CIFAR-100 dataset with 32×32 input images. Specifically, 3×3 convolutional layers are used instead of the original 7×7, an activation function, and batch normalization is applied to improve training stability. The network contains eight residual blocks with 64, 128, 256, and 512 filters. Finally, the network ends with global average pooling, followed a fully connected layer with 128 units and a softmax output layer with 100 units for multi-class classification.

4.1.2. Text classification models

- **Long Short-Term Memory (LSTM):** In this experiment, we adopt the LSTM architecture proposed by [Setyanto et al. \(2022\)](#). The model consists of an Embedding layer, a single LSTM layer with 64 units, a fully connected Dense layer with 32 units followed by an activation function and a dropout layer, and a final Dense layer with a sigmoid activation for output prediction.
- **Bidirectional LSTM (BiLSTM):** In this experiment, we employ a BiLSTM model for sequence data. The architecture consists of an Embedding layer, a BiLSTM layer with 32 units, dropout layers, a Dense layer with 32 units followed by an activation function, and a final Dense layer with softmax activation for output prediction.

4.2. Dataset description and preprocessing techniques

The benchmark datasets used in this study are divided into two categories: image and text classification. The first category includes standard benchmark datasets for image classification tasks, including MNIST ([Deng, 2012](#)), Fashion-MNIST ([Xiao et al., 2017](#)), and CIFAR-100 ([Krizhevsky & Hinton, 2009](#)). The second category consists of benchmark datasets for text classification tasks, including IMDB movie reviews ([Team, 2020a](#)) and Reuters newswire topics datasets ([Team, 2020b](#)). [Table 3](#) provides the detailed properties of each dataset used in our experiments.

To ensure data consistency and optimize learning efficiency, appropriate preprocessing was applied to each dataset. For the image datasets (MNIST, Fashion-MNIST, and CIFAR-100), this included normalization, reshaping, one-hot label encoding, and train-test splitting. For text datasets, IMDB reviews were tokenized, padded, and split, while

the pre-tokenized Reuters Newswire dataset was padded to a fixed sequence length and its labels one-hot encoded for multi-class classification.

4.3. Experimental settings

The experiments were conducted on Kaggle's cloud platform equipped with Nvidia T4 Tensor Core GPUs ([Kaggle Inc., 2017](#)). The proposed MishReLU activation function was implemented using Python and machine learning libraries such as TensorFlow ([Chollet, 2018](#)). The best hyperparameters for each dataset were selected as part of the hyperparameter tuning using the grid search ([Ippolito, 2022; Zahedi et al., 2021](#)), with learning rate values adopted from previous studies and validated through five random seeds, ensuring stable training and optimal validation performance. The hyperparameters used for all training experiments are summarized in [Table 4](#).

4.4. Performance metrics

The performance of the MishReLU activation function is evaluated using four standard metrics: Accuracy, Precision, Recall, and F1-score. The definitions of these metrics are provided below. To ensure robustness, all results are reported as mean ± standard deviation over multiple experimental runs with five different random seeds to ensure the reliability and stability of the evaluation.

- **Accuracy:** The accuracy measures the overall classification performance by computing the ratio of correctly predicted samples to the total number of samples.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

- **Precision:** Precision represents the fraction of correctly identified positive instances out of all instances classified as positive by the model.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (5)$$

- **Recall:** Recall refers to the fraction of correctly detected positive instances among the total number of actual positive instances in the dataset.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (6)$$

- **F1-score:** The F1-score provides a balanced measure of test performance by incorporating both precision and recall into a single score.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (7)$$

5. Results and discussions

5.1. Image classification results

[Tables 5–7](#) present a performance comparison between the MishReLU activation function and several traditional activation functions for image classification tasks on the Fashion-MNIST, MNIST, and CIFAR-100 datasets. The evaluation is based on the mean of accuracy, precision, recall, and F1 score, along with their corresponding standard deviations. According to the experimental results reported in [Table 5](#), the best overall performance is achieved using the MishReLU with a learning rate of 0.0001. In particular, as summarized in [Table 5](#), MishReLU has demonstrated the highest mean accuracy, precision, recall, and F1 score on the Fashion-MNIST dataset, with values of 88.69%, 88.93%, 88.69%, and 88.72%, respectively. Moreover, MishReLU shows a relatively low standard deviation (0.0041) at a learning rate of 0.0001, indicating consistent performance across five random seeds.

The experimental results on the MNIST dataset using the CNN model are listed in [Table 6](#). The MishReLU has produced the best performance with a learning rate of 0.001. In particular, as summarized in

Table 3
A summary of the benchmark datasets.

Dataset	Task	Training	Testing	Total	# Of Classes
Fashion-MNIST	Image Classification	60,000	10,000	70,000	10
MNIST	Image Classification	60,000	10,000	70,000	10
CIFAR-100	Image Classification	50,000	10,000	60,000	100
Reuters Newswire	Text Classification	8,982	2,246	11,228	46
IMDB	Text Classification	25,000	25,000	50,000	2

Table 4
The hyperparameters used throughout all training experiments.

Component	MLP	CNN	ResNet-18	LSTM	BiLSTM
Batch Size	64	256	256	256	128
Epochs	30	30	20	10	15
Optimizer	Nadam	Rmsprop	Adam	Nadam	Adam
Embedding	–	–	–	128	32
Dropout	0.2	0.25, 0.5	–	0.2, 0.3	0.2, 0.3
Loss	Categorical Cross-Entropy	Categorical Cross-Entropy	Categorical Cross-Entropy	Binary Cross-Entropy	Categorical Cross-Entropy

Table 6, MishReLU has demonstrated the highest mean accuracy, precision, recall, and F1 score on the MNIST dataset, with values of 99.47%, 99.46%, 99.46%, and 99.46%, respectively. Moreover, MishReLU exhibits a lower standard deviation (0.0003) at a learning rate of 0.001 compared to Mish, indicating more stable training behavior.

Performance of ResNet-18 model on the CIFAR-100 dataset with different activation functions is as summarized in **Table 7**. The ResNet-18 model with MishReLU outperforms other activation functions with a learning rate of 0.001. It has achieved the highest mean accuracy, precision, recall, and F1 score on the CIFAR100 dataset, with values of 57.56%, 61.26%, 57.65%, and 57.36%, respectively. Moreover, MishReLU exhibits a lower standard deviation (0.0161) compared to Mish, indicating more stable training behavior. These results demonstrate that MishReLU provides an improved balance between high accuracy and relatively low standard deviation, consistently outperforming the other activation functions across the evaluated datasets.

5.2. Text classification results

Tables 8–9 show the details of the experimental results of the MishReLU activation function and several traditional activation functions for text classification tasks using the IMDB, and the Reuters Newswire datasets. The evaluation is based on the mean of accuracy, precision, recall, and F1 score, along with their corresponding standard deviations. According to the experimental results reported in **Tables 8–9**, MishReLU has demonstrated the best performance across all models with a learning rate of 0.01. The experimental results on the IMDB dataset using the LSTM model are listed in **Table 8**. MishReLU

has demonstrated the highest mean accuracy, precision, recall, and F1 score on the IMDB dataset, with values of 87.92%, 87.94%, 87.92%, and 87.92%, respectively. Moreover, MishReLU exhibits a lower standard deviation (0.0021) at a learning rate of 0.01 compared to ReLU, indicating more stable behavior.

The experimental results on the Reuters Newswire dataset using the BiLSTM model are listed in **Table 9**. MishReLU has demonstrated the highest mean accuracy, precision, recall, and F1 score on the Reuters Newswire dataset, with values of 67.67%, 66.12%, 67.67%, and 66.33%, respectively. Moreover, MishReLU exhibits a lower standard deviation (0.0098) at a learning rate of 0.01 compared to other activation functions, indicating more stable training behavior. These results demonstrate that MishReLU provides an improved balance between the accuracy and the stability of the other activation functions, demonstrating its effectiveness as an activation function for the LSTM and BiLSTM models on the IMDB and Reuters Newswire datasets.

Statistical comparisons were conducted using the Friedman test for global performance evaluation, followed by the Wilcoxon signed-rank test as a post-hoc analysis to compare the MishReLU activation function with other functions across all learning rates. According to the Friedman test, statistically significant differences were observed among the activation functions in the MLP and ResNet-18 models at learning rates of 0.01 and 0.001, and in the LSTM model specifically at a learning rate of 0.01 ($p < 0.05$). However, the Wilcoxon signed-rank test indicated that no statistically significant pairwise differences were found ($p > 0.05$).

To investigate why the MishReLU activation function has a superior performance, **Table 10** shows a set of gradient and activation output his-

Table 5
Performance metrics for different activation functions of MLP on the FASHION-MNIST dataset.

Metric(%)	LR	MishReLU	ReLU	Mish	ELU	LeakyReLU	Selu
Accuracy	0.01	86.97 ± 0.0072	87.00 ± 0.0033	87.06 ± 0.0054	87.21 ± 0.0060	87.17 ± 0.0055	87.52 ± 0.0059
	0.001	88.65 ± 0.0057	87.43 ± 0.0067	88.62 ± 0.0047	87.79 ± 0.0059	87.98 ± 0.0021	87.69 ± 0.0088
	0.0001	88.69 ± 0.0041	88.27 ± 0.0038	88.61 ± 0.0036	87.93 ± 0.0040	88.24 ± 0.0024	87.53 ± 0.0042
Precision	0.01	87.38 ± 0.0046	87.38 ± 0.0033	87.30 ± 0.0042	87.67 ± 0.0036	87.66 ± 0.0046	87.85 ± 0.0040
	0.001	88.81 ± 0.0036	87.69 ± 0.0045	88.78 ± 0.0031	88.02 ± 0.0044	88.28 ± 0.0024	88.01 ± 0.0061
	0.0001	88.93 ± 0.0022	88.45 ± 0.0021	88.86 ± 0.0011	88.17 ± 0.0020	88.34 ± 0.0013	87.72 ± 0.0022
Recall	0.01	86.97 ± 0.0072	87.00 ± 0.0033	87.06 ± 0.0054	87.21 ± 0.0060	87.17 ± 0.0055	87.52 ± 0.0059
	0.001	88.65 ± 0.0057	87.43 ± 0.0067	88.62 ± 0.0047	87.79 ± 0.0059	87.98 ± 0.0021	87.69 ± 0.0088
	0.0001	88.69 ± 0.0041	88.27 ± 0.0038	88.61 ± 0.0036	87.93 ± 0.0040	88.24 ± 0.0024	87.53 ± 0.0042
F1 score	0.01	86.96 ± 0.0068	87.01 ± 0.0039	87.00 ± 0.0060	87.20 ± 0.0055	87.19 ± 0.0058	87.48 ± 0.0056
	0.001	88.62 ± 0.0053	87.41 ± 0.0060	88.61 ± 0.0046	87.77 ± 0.0057	88.01 ± 0.0024	87.65 ± 0.0087
	0.0001	88.72 ± 0.0038	88.21 ± 0.0034	88.63 ± 0.0032	87.89 ± 0.0038	88.21 ± 0.0020	87.47 ± 0.0040

Table 6
Performance metrics for different activation functions of CNN on the MNIST dataset.

Metric(%)	LR	MishReLU	ReLU	Mish	ELU	LeakyReLU	Selu
Accuracy	0.01	99.34 ± 0.0006	99.36 ± 0.0007	99.38 ± 0.0004	99.35 ± 0.0006	99.38 ± 0.0007	99.40 ± 0.0004
	0.001	99.47 ± 0.0003	99.43 ± 0.0004	99.45 ± 0.0003	99.47 ± 0.0004	99.44 ± 0.0006	99.45 ± 0.0005
	0.0001	99.38 ± 0.0002	99.37 ± 0.0003	99.37 ± 0.0004	99.34 ± 0.0002	99.36 ± 0.0005	99.25 ± 0.0004
Precision	0.01	99.34 ± 0.0006	99.37 ± 0.0007	99.38 ± 0.0004	99.35 ± 0.0006	99.38 ± 0.0007	99.39 ± 0.0004
	0.001	99.46 ± 0.0004	99.43 ± 0.0004	99.45 ± 0.0003	99.47 ± 0.0004	99.44 ± 0.0006	99.45 ± 0.0005
	0.0001	99.38 ± 0.0002	99.37 ± 0.0003	99.37 ± 0.0004	99.34 ± 0.0003	99.36 ± 0.0005	99.26 ± 0.0004
Recall	0.01	99.34 ± 0.0006	99.35 ± 0.0007	99.37 ± 0.0004	99.35 ± 0.0005	99.37 ± 0.0008	99.39 ± 0.0004
	0.001	99.46 ± 0.0003	99.42 ± 0.0004	99.44 ± 0.0003	99.46 ± 0.0004	99.43 ± 0.0006	99.45 ± 0.0006
	0.0001	99.37 ± 0.0002	99.36 ± 0.0004	99.36 ± 0.0004	99.33 ± 0.0003	99.35 ± 0.0005	99.24 ± 0.0004
F1 score	0.01	99.34 ± 0.0006	99.36 ± 0.0007	99.37 ± 0.0004	99.35 ± 0.0005	99.37 ± 0.0008	99.39 ± 0.0004
	0.001	99.46 ± 0.0004	99.42 ± 0.0004	99.44 ± 0.0003	99.47 ± 0.0004	99.44 ± 0.0006	99.45 ± 0.0006
	0.0001	99.38 ± 0.0002	99.36 ± 0.0003	99.37 ± 0.0004	99.33 ± 0.0003	99.35 ± 0.0005	99.25 ± 0.0004

Table 7
Performance metrics for different activation functions of ResNet-18 on the CIFAR-100 dataset.

Metric (%)	LR	MishReLU	ReLU	Mish	ELU	Leaky ReLU	SeLU
Accuracy	0.01	45.07 ± 0.1395	40.23 ± 0.0582	52.06 ± 0.0121	49.27 ± 0.0107	49.67 ± 0.0234	37.10 ± 0.0902
	0.001	57.65 ± 0.0161	56.94 ± 0.0109	56.92 ± 0.0234	55.14 ± 0.0084	55.53 ± 0.0170	52.33 ± 0.0063
	0.0001	52.39 ± 0.0037	48.75 ± 0.0096	53.13 ± 0.0080	52.08 ± 0.0013	49.97 ± 0.0065	49.57 ± 0.0055
Precision	0.01	48.97 ± 0.1566	44.16 ± 0.0576	56.18 ± 0.0178	54.68 ± 0.0092	54.09 ± 0.0132	44.58 ± 0.1035
	0.001	61.26 ± 0.0146	62.13 ± 0.0143	60.61 ± 0.0218	59.17 ± 0.0096	60.94 ± 0.0139	56.52 ± 0.0072
	0.0001	55.91 ± 0.0079	52.82 ± 0.0067	55.92 ± 0.0063	55.38 ± 0.0053	53.65 ± 0.0045	52.23 ± 0.0060
Recall	0.01	45.07 ± 0.1395	40.23 ± 0.0582	52.06 ± 0.0121	49.27 ± 0.0107	49.67 ± 0.0234	37.10 ± 0.0902
	0.001	57.65 ± 0.0161	56.94 ± 0.0109	56.92 ± 0.0234	55.14 ± 0.0084	55.53 ± 0.0170	52.33 ± 0.0063
	0.0001	52.39 ± 0.0037	48.75 ± 0.0096	53.13 ± 0.0080	52.08 ± 0.0013	49.97 ± 0.0065	49.57 ± 0.0055
F1 score	0.01	44.25 ± 0.1515	38.92 ± 0.0662	51.81 ± 0.0150	48.38 ± 0.0093	49.32 ± 0.0223	35.66 ± 0.0956
	0.001	57.36 ± 0.0174	56.61 ± 0.0121	56.53 ± 0.0253	54.76 ± 0.0095	55.28 ± 0.0174	51.78 ± 0.0071
	0.0001	51.96 ± 0.0041	48.36 ± 0.0098	52.79 ± 0.0081	51.53 ± 0.0025	49.36 ± 0.0063	48.83 ± 0.0047

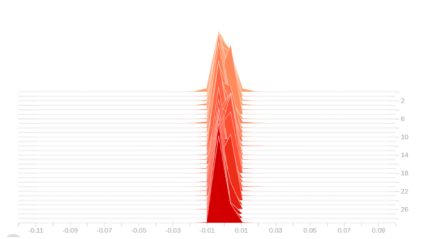
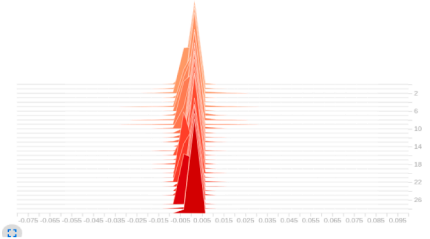
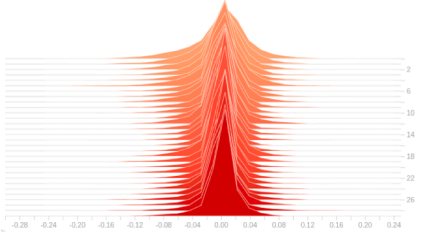
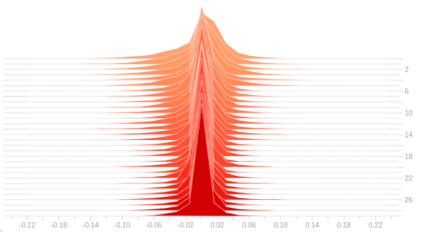
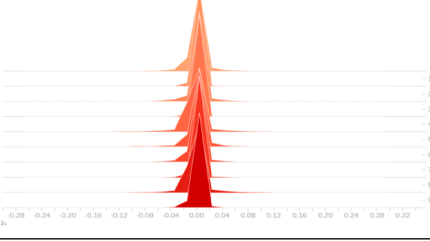
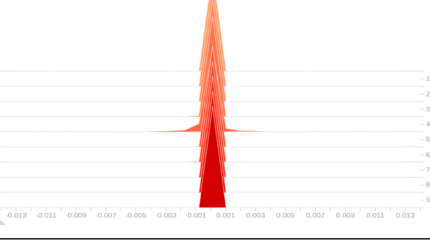
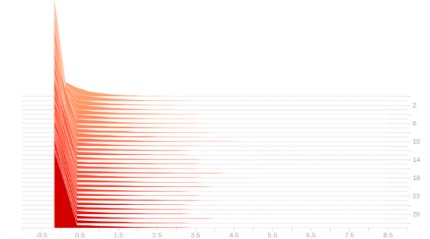
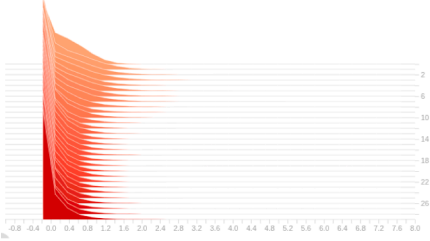
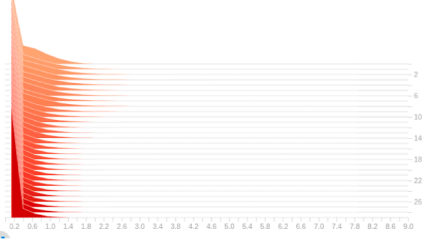
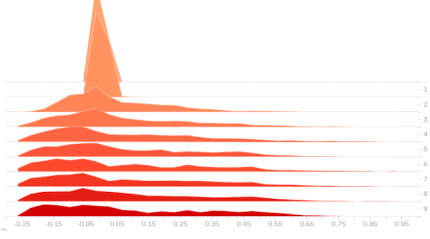
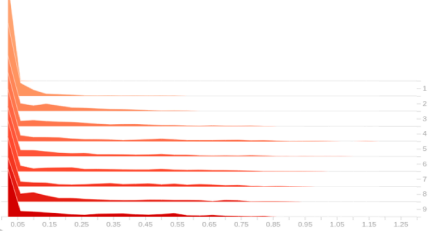
Table 8
Performance metrics for different activation functions of the LSTM Model on the IMDB dataset.

Metric	LR	MishReLU	ReLU	Mish	ELU	LeakyReLU	Selu
Accuracy	0.01	87.92±0.0021	87.56±0.0026	87.68±0.0012	87.52±0.0028	87.60±0.0030	87.31±0.0032
	0.001	86.85±0.0061	87.07±0.0042	84.69±0.0491	86.41±0.0094	87.50±0.0031	86.30±0.0043
	0.0001	86.74±0.0029	86.73±0.0037	86.73±0.0023	86.59±0.0041	87.13±0.0034	86.19±0.0030
Precision	0.01	87.94±0.0021	87.59±0.0027	87.70±0.0012	87.57±0.0027	87.70±0.0023	87.37±0.0027
	0.001	86.89±0.0059	87.11±0.0040	84.71±0.0488	86.45±0.0093	87.51±0.0031	86.34±0.0046
	0.0001	86.76±0.0029	86.78±0.0032	86.76±0.0023	86.63±0.0041	87.14±0.0034	86.24±0.0029
Recall	0.01	87.92±0.0021	87.56±0.0026	87.68±0.0012	87.52±0.0028	87.60±0.0030	87.31±0.0032
	0.001	86.85±0.0061	87.07±0.0042	84.69±0.0491	86.41±0.0094	87.50±0.0031	86.30±0.0043
	0.0001	86.74±0.0029	86.73±0.0037	86.73±0.0023	86.59±0.0041	87.13±0.0034	86.19±0.0030
F1 score	0.01	87.92±0.0021	87.56±0.0025	87.68±0.0012	87.51±0.0028	87.60±0.0031	87.31±0.0032
	0.001	86.84±0.0062	87.07±0.0042	84.68±0.0492	86.41±0.0094	87.50±0.0031	86.30±0.0042
	0.0001	86.74±0.0029	86.72±0.0037	86.73±0.0024	86.59±0.0041	87.13±0.0034	86.19±0.0031

Table 9
Performance metrics for different activation functions of the BiLSTM Model on the Reuters Newswire dataset.

Metric	LR	MishReLU	ReLU	Mish	ELU	LeakyReLU	Selu
Accuracy	0.01	67.67±0.0098	65.34±0.0143	66.22±0.0161	66.87±0.0109	65.22±0.0157	66.17±0.0165
	0.001	57.27±0.0215	56.20±0.0351	56.43±0.0287	57.36±0.0181	57.80±0.0201	59.07±0.0160
	0.0001	53.13±0.0117	53.22±0.0178	51.50±0.0160	53.00±0.0163	53.01±0.0129	52.97±0.0032
Precision	0.01	66.12±0.0148	62.33±0.0144	65.26±0.0237	66.07±0.0167	63.54±0.0224	65.26±0.0296
	0.001	53.55±0.0206	51.13±0.0289	49.01±0.0578	52.66±0.0208	52.19±0.0125	54.36±0.0246
	0.0001	41.55±0.0258	41.19±0.0329	39.18±0.0164	41.76±0.0307	41.03±0.0169	39.58±0.0111
Recall	0.01	67.67±0.0098	65.34±0.0143	66.22±0.0161	66.87±0.0109	65.22±0.0157	66.17±0.0165
	0.001	57.27±0.0215	56.20±0.0351	56.43±0.0287	57.36±0.0181	57.80±0.0201	59.07±0.0160
	0.0001	53.13±0.0117	53.22±0.0178	51.50±0.0160	53.00±0.0163	53.01±0.0129	52.97±0.0032
F1 score	0.01	66.33±0.0119	63.11±0.0138	65.09±0.0184	65.74±0.0136	63.61±0.0180	64.90±0.0228
	0.001	54.44±0.0195	52.49±0.0304	51.39±0.0478	53.94±0.0153	53.83±0.0129	55.68±0.0174
	0.0001	44.19±0.0237	44.14±0.0295	42.26±0.0085	44.18±0.0280	43.85±0.0196	43.04±0.0042

Table 10
Gradient and activation output histograms for each activation function.

Gradient Graph	Histogram	MishReLU	ReLU
Gradient	log-normal-histograms		
			
			
Activation Output	log-normal-histograms		
			
			

tograms graphs for MishReLU and ReLU during the training of multiple models, including MLP, CNN, and LSTM. The figures in Table 10 are 3D gradient histograms that show how the gradients of weights and outputs for the activation layer are distributed during training. The figures include three axes: time (x-axis), value (y-axis), and frequency/density of values (z-axis). Instead of commonly used time units such as minutes or seconds, the unit of the time axis (x-axis) is an epoch, which represents the NN's training progress. Figures show lighter histograms for older data at earlier training epochs and darker histograms for newer data at recent epochs.

The figures in Table 10 were taken directly from TensorBoard (Google, 2020), a tool developed by Google for visualizing neural network training. Tensorboard is widely used in the field of AI research for validating theories and hypotheses. Table 10 shows how the MishReLU activation function addresses the dying ReLU problem, as observed from the gradient and activation output histograms. The gradient histogram for MishReLU is smooth and spans a wider range than ReLU, whose gradients are narrowly concentrated and close to zero. The activation output histogram for MishReLU is continuous, covering both positive and negative values. In contrast, the activation output histogram for the ReLU function shows a sharp spike at zero. Once the gradient becomes zero, the model struggles to recover, and affected neurons become inactive, an indication of the dying ReLU problem.

6. Conclusions

DNN performance is significantly influenced by the choice of activation functions, which enable the network to learn and predict complex relationships within the data. While sigmoid and tanh activation functions are known to suffer from the exploding and vanishing gradient problems, the ReLU activation function was introduced to overcome this issue. However, it fails to produce the correct outputs for negative inputs, as it returns zero for all values in the negative region (the "dying ReLU" problem). To address this issue, we propose a hybrid activation function, MishReLU, which combines the properties of Mish and ReLU to overcome both the vanishing gradient and negative value problems. Experiments were conducted on the Fashion-MNIST, MNIST, CIFAR-100, IMDb Movie Review, and Reuters Newswire datasets using models such as MLP, CNN, ResNet-18, LSTM, and BiLSTM with the MishReLU activation function. The results show that MishReLU achieves superior performance compared to other activation functions.

CRedit authorship contribution statement

Naseem Al-nawi: Conceptualization, Methodology, Software, Writing - original draft. Alia Madain: Conceptualization, Methodology, Writing - review & editing. Malak Abdullah: Conceptualization, Methodology, Writing - review & editing.

Data availability

The data used in this study has been made available by the data owners and can be downloaded via the links included in the manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Absur, M. N., Nasif, K. F. A., Saha, S., & Nova, S. N. (2024). Revolutionizing image recognition: Next-generation CNN architectures for handwritten digits and objects. In *2024 IEEE symposium on wireless technology applications (ISWTA)* (pp. 173–178). <https://doi.org/10.1109/ISWTA62130.2024.10651815>

- Agatonovic-Kustrin, S., & Beresford, R. (2000). Basic concepts of artificial neural network (ANN) modeling and its application in pharmaceutical research. *Journal of Pharmaceutical and Biomedical Analysis*, 22(5), 717–727. [https://doi.org/10.1016/S0731-7085\(99\)00272-1](https://doi.org/10.1016/S0731-7085(99)00272-1)
- Agostinelli, F., Anderson, M. R., & Lee, H. (2013). Adaptive multi-column deep neural networks with application to robust image denoising. *Advances in Neural Information Processing Systems*, 26, 1493–1501.
- Airlangga, G., Bata, J., Adi Nugroho, O. I., & Lim, B. H. P. (2025). Hybrid CNN-LSTM model with custom activation and loss functions for predicting fan actuator states in smart greenhouses. *AgriEngineering*, 7(4). <https://doi.org/10.3390/agriengineering7040118>
- Alkhoully, A. A., Mohammed, A., & Hefny, H. A. (2021). Improving the performance of deep neural networks using two proposed activation functions. *IEEE Access*, 9, 82249–82271. <https://doi.org/10.1109/ACCESS.2021.3085855>
- Ankalaki, S., & Thippeswamy, M. N. (2024). A novel optimized parametric hyperbolic tangent swish activation function for 1d-CNN: application of sensor-based human activity recognition and anomaly detection. *Multimedia Tools and Applications*, 83(22), 61789–61819. <https://doi.org/10.1007/s11042-023-15766-3>
- Anton, H., Bivens, I. C., & Davis, S. (2012). *Calculus*, 10th edition. Wiley. <https://books.google.jo/books?id=xSkcAAAAQBAJ>
- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Berlin, Germany: Springer.
- Chollet, F. (2018). Keras: The python deep learning library. In *Astrophysics source code library (ASCL)* (p. 1806).
- Clevert, D. A., Unterthiner, T., & Hochreiter, S. (2015). Fast and accurate deep network learning by exponential linear units (ELUs).
- Deng, L. (2012). The MNIST database of handwritten digit images for machine learning research [best of the web]. *IEEE Signal Processing Magazine*, 29(6), 141–142.
- Dubey, S. R., Singh, S. K., & Chaudhuri, B. B. (2022). Activation functions in deep learning: A comprehensive survey and benchmark. *arXiv:2109.14545*
- Duch, W., & Jankowski, N. (1999). Survey of neural transfer functions. *Neural Computing Surveys*, 2(1), 163–212.
- Emanuel, R. H. K., Docherty, P. D., Lunt, H., & Möller, K. (2024). The effect of activation functions on accuracy, convergence speed, and misclassification confidence in CNN text classification: a comprehensive exploration. *The Journal of Supercomputing*, 80(1), 292–312. <https://doi.org/10.1007/s11227-023-05441-7>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <http://www.deeplearningbook.org>.
- Google (2020). TensorBoard | tensorflow. <https://www.tensorflow.org/tensorboard>. Accessed: 2020-08-03.
- Goyal, M., Goyal, R., Reddy, P. V., & Lall, B. (2019). Activation functions. Springer International Publishing, In *Deep Learning: Algorithms and Applications* (pp. 1–30), 865. https://doi.org/10.1007/978-3-030-31760-7_1
- Hasan, M. M., Hossain, M. A., Srizon, A. Y., & Sayeed, A. (2023). TaLU: A hybrid activation function combining tanh and rectified linear unit to enhance neural networks. *arXiv:2305.04402*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *2016 IEEE conference on computer vision and pattern recognition (CVPR)* (pp. 770–778). <https://doi.org/10.1109/CVPR.2016.90>
- Heaton, J., Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org/>.
- Ippolito, P. P. (2022). Hyperparameter tuning. In R. Egger (Ed.), *Applied data science in tourism: Interdisciplinary approaches, methodologies, and applications* (pp. 231–251). Cham: Springer International Publishing. https://doi.org/10.1007/978-3-030-88389-8_12
- Ji, Y., Zhu, Y., Li, Z., Chen, J., Kong, Y., & Chen, J. (2025). Enhancing out-of-distribution detection through dynamic activation function. In *Icassp 2025 - 2025 IEEE international conference on acoustics, speech and signal processing (icassp)* (pp. 1–5). <https://doi.org/10.1109/ICASSP49660.2025.10890331>
- Kaggle Inc. (2017). Explore and run machine learning code with kaggle notebooks. <https://www.kaggle.com/kernels>.
- Kalim, H., Chug, A., & Singh, A. P. (2024). Enhancement of neural network performance with the use of two novel activation functions: modexp and modexpm. *Optical Memory and Neural Networks*, 33(3), 286–301. <https://doi.org/10.3103/S1060992X24700152>
- Kiliçarslan, S.C.M., & (2021). Rsigelu: A nonlinear activation function for deep neural networks. *Expert Systems with Applications*, 9, 114805.
- Közkurt, C., Diker, A., Elen, A., Kiliçarslan, S., Dönmez, E., & Demir, F. B. (2024). Trish: An efficient activation function for CNN models and analysis of its effectiveness with optimizers in diagnosing glaucoma. *The Journal of Supercomputing*, 80(11), 15485–15516. <https://doi.org/10.1007/s11227-024-06057-1>
- Krizhevsky, A., & Hinton, G. (2009). Learning multiple layers of features from tiny images. Technical Report TR-2009. University of Toronto. Technical Report.
- LeCun, Y., Bottou, L., Orr, G. B., & Müller, K. R. (2002). Efficient backprop. In *Neural networks: Tricks of the trade*, pp. 9–50. Berlin, Heidelberg: Springer Berlin Heidelberg. https://doi.org/10.1007/3-540-49430-8_2
- Li, X., Xia, Z., & Zhang, H. (2025). Cauchy activation function and XNet. *Neural Networks*, 188, 107375. <https://doi.org/10.1016/j.neunet.2025.107375>
- Liu, K., Kang, G., Zhang, N., & Hou, B. (2018a). Breast cancer classification based on fully-connected layer first convolutional neural networks. *IEEE Access*, 6, 23722–23732. <https://doi.org/10.1109/ACCESS.2018.2817593>
- Liu, Y., Liu, S., Wang, Y., Lombardi, F., & Han, J. (2018b). A stochastic computational multi-layer perceptron with backward propagation. *IEEE Transactions on Computers*, 67(9), 1273–1286. <https://doi.org/10.1109/TC.2018.2817237>
- Liu, Y., Zhang, J., Gao, C., Qu, J., & Ji, L. (2019). Natural-logarithm-rectified activation function in convolutional neural networks. In *Proceedings of the 2019 IEEE 5th international conference on computer and communications (ICCC)*, Chengdu, China (pp. 2000–2008).

- Maas, A. L., Hannun, A. Y., & Ng, A. Y. (2013). Rectifier nonlinearities improve neural network acoustic models. In *Proceedings of the 30th international conference on machine learning (ICML)* (pp. 1–8).
- Madhu, G., Kautish, S., Alnowibet, K. A., Zawbaa, H. M., & Mohamed, A. W. (2023). Nipuna: A novel optimizer activation function for deep neural networks. *Axioms*, 12(3). <https://doi.org/10.3390/axioms12030246>
- Misra, D. (2020). Mish: A self regularized non-monotonic activation function. *arXiv:1908.08681*.
- Mukhamediev, R. I. (2024). State-of-the-art results with the fashion-MNIST dataset. *Mathematics*, 12(20). <https://doi.org/10.3390/math12203174>
- Nag, S., Mukherjee, A., & Kundu, R. (2023). SERF Towards better training of deep neural networks using log-softplus ERror activation function. *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 47, 5324–5333.
- Nandi, A., Jana, N. D., & Das, S. (2020). Improving the performance of neural networks with an ensemble of activation functions. In *2020 international joint conference on neural networks (IJCNN)* (pp. 1–7). <https://doi.org/10.1109/IJCNN48605.2020.9207277>
- Nwankpa, C., Ijomah, W., Gachagan, A., & Marshall, S. (2018). Activation functions: Comparison of trends in practice and research for deep learning. *arXiv:1811.03378*.
- Ramachandran, P., Zoph, B., & Le, Q. V. (2017). Searching for activation functions. *arXiv:1710.05941*
- Rasamoelina, A. D., Adjailia, F., & Sinčák, P. (2020). A review of activation function for artificial neural network. In *2020 IEEE 18th world symposium on applied machine intelligence and informatics (SAMI)* (pp. 281–286). <https://doi.org/10.1109/SAMI48414.2020.9108717>
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. <https://doi.org/10.1038/323533a0>
- Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural Networks*, 61, 85–117. <https://doi.org/10.1016/j.neunet.2014.09.003>
- Setyanto, A., Laksito, A., Alarfaj, F., Alreshoodi, M., Kusri, Oyong, I., Hayaty, M., Alomair, A., Almusallam, N., & Kurniasari, L. (2022). Arabic language opinion mining based on long short-term memory (LSTM). *Applied Sciences*, 12(9). <https://doi.org/10.3390/app12094140>
- Sharma, S., Sharma, S., & Athaiya, A. (2017). Activation functions in neural networks. *Towards Data Sci*, 6(12), 310–316.
- Subramanian, B., Jeyaraj, R., Ugli, R. A. A., & Kim, J. (2024). Apalu: A trainable, adaptive activation function for deep learning networks. *arXiv:2402.08244*
- Szandala, T. (2021). Review and comparison of commonly used activation functions for deep neural networks. In *Bio-inspired Neurocomputing*, pp. 203–224. Singapore: Springer Singapore. https://doi.org/10.1007/978-981-15-5495-7_11
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2014). Going deeper with convolutions. *newblock In Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)* (pp. 1–9) *arXiv:1409.4842*.
- Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)* (pp. 4700–4708). *arXiv:1608.06993*.
- Sára Puzstaházi, L., Eigner, G., & Csiszár, O. (2024). Parametric activation functions for neural networks: A tutorial survey. *IEEE Access*, 12, 168626–168644. <https://doi.org/10.1109/ACCESS.2024.3474574>
- Team, K. (2020a). Keras documentation: IMDB movie review sentiment classification dataset. Accessed on November 28, 2020 <https://keras.io/api/datasets/imdb/>.
- Team, K. (2020b). Keras documentation: Reuters newswire classification dataset. Accessed on September 2, 2020 <https://keras.io/api/datasets/reuters/>.
- Turian, J., Bergstra, J., & Bengio, Y. (2009). Quadratic features and deep architectures for chunking. In *Proceedings of human language technologies: The 2009 annual conference of the north american chapter of the association for computational linguistics* (pp. 245–248). Association for Computational Linguistics (vol. *Companion Volume*). <https://dl.acm.org/citation.cfm?id=1558160>.
- Wang, X., Qin, Y., Wang, Y., Xiang, S., & Chen, H. (2019). ReLTanh: An activation function with vanishing gradient resistance for SAE-based DNNs and its application to rotating machinery fault diagnosis. *Neurocomputing*, 363, 88–98. <https://doi.org/10.1016/j.neucom.2019.07.078>
- Wang, X., Ren, H., & Wang, A. (2022). Smish: A novel activation function for deep learning methods. *Electronics*, 11(4). <https://doi.org/10.3390/electronics11040540>
- Wu, L., Wang, S., Fang, L., & Du, H. (2021). Mmrelu: A simple and smooth activation function with high convergence speed. In *2021 7th international conference on computer and communications (ICCC)* (pp. 1444–1448). <https://doi.org/10.1109/ICCC54389.2021.9674529>
- Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv:1708.07747*
- Xinyu, L. (2020). Tanhexp: A smooth activation function with high convergence speed for lightweight neural networks. *arXiv:2006.05792*
- Xu, Z., Tang, B., Zhang, X., Leong, J. F., Pan, J., Hooda, S., Zamburg, E., & Thean, A. V.-Y. (2022). Reconfigurable nonlinear photonic activation function for photonic neural network based on non-volatile opto-resistive RAM switch. *Light: Science & Applications*, 11(1), 288. <https://doi.org/10.1038/s41377-022-00976-5>
- Yu, Y., Adu, K., Tashi, N., Anokye, P., Wang, X., & Ayidzoe, M. A. (2020). Rmaf: Relu-memristor-like activation function for deep learning. *IEEE Access*, 8, 72727–72741. <https://doi.org/10.1109/ACCESS.2020.2987829>
- Zahedi, L., Mohammadi, F. G., Rezapour, S., Ohland, M. W., & Amini, M. H. (2021). Search algorithms for automated hyper-parameter tuning. *arXiv:2104.14677*.
- Zeiler, M. D., & Fergus, R. (2014). Visualizing and understanding convolutional networks. In *European conference on computer vision (ECCV)* (pp. 818–833).
- Zhang, G., & Li, H. (2018). Effectiveness of scaled exponentially-regularized linear units (SERLUs). *arXiv:1807.10117*.